RETO DE HACKATHON

El Guardián

Detecta cuándo una inteligencia artificial se inventa las cosas

Temas: inteligencia artificial · procesamiento de lenguaje natural · análisis de datos

Para quién: estudiantes que se manejen con Python. No hace falta experiencia previa en IA.

Qué necesitas: un portátil con Python 3. Nada más.

1. El problema: las IA se inventan cosas

Los asistentes de inteligencia artificial ya responden preguntas en bancos, hospitales, administraciones y empresas de todo tipo. Para no inventar, se les da una documentación de referencia (las **fuentes**) y se les pide que respondan basándose en ella. Es lo que en el sector se llama RAG (*retrieval-augmented generation*).

Aun así, a veces fallan: responden con total seguridad algo que sus fuentes **no dicen**. Cambian una cifra, atribuyen un dato a quien no es, añaden una causa que nadie ha mencionado o convierten «algunos» en «todos». A esto se le llama **alucinación**, y es hoy uno de los principales motivos por los que las empresas no se atreven a poner estos asistentes delante de sus clientes.

La solución que usa la industria es un **guardián** (*guardrail*): un sistema que revisa cada respuesta antes de que llegue al usuario y da la alarma si detecta que se ha inventado algo. Eso es lo que vais a construir.

> **El reto en una frase**
>
> **Construid un detector que, ante la respuesta de una IA y sus fuentes, diga si la IA se ha inventado algo.**

2. Qué vais a construir

Un programa en Python, el **Guardián**, que recibe un caso y devuelve un número entre 0 y 1: la probabilidad de que la respuesta contenga alguna alucinación.

Recibe	Devuelve
pregunta: lo que preguntó el usuario **fuentes**: la documentación que tenía la IA **respuesta**: lo que contestó la IA **señales**: lo que opinan dos modelos de IA sobre cada frase (ver apartado 4)	Un número entre **0** (la respuesta es fiel a las fuentes) y **1** (la respuesta se inventa algo). A partir de 0,5 se considera que el bot dice «alucina».

Es una clase de Python con un único método:

```
class Guardian:
    def score(self, caso) -> float:
        # caso["pregunta"], caso["fuentes"], caso["respuesta"], caso["senales"]
        # devolved un número entre 0 (fiel) y 1 (alucina)
        ...
```

> **Requisitos**
>
> **No hay que instalar nada más que Python 3** (versión 3.9 o superior). No se ejecuta ningún modelo de IA en vuestro portátil: el trabajo pesado ya lo hemos hecho nosotros y viene incluido en cada caso. Funciona en cualquier ordenador.

3. Un caso de ejemplo

Así es un caso real del conjunto de entrenamiento. Leed las fuentes y la respuesta antes de mirar la solución.

Fuentes
1. La agrupación de voluntarios de protección civil de Montejo cuenta con 46 miembros. 2. En 2025 intervino en 3 incendios forestales y en 19 eventos públicos. 3. Sus miembros deben completar un curso inicial de 60 horas. 4. La agrupación dispone de dos vehículos, uno de ellos con depósito de agua de 600 litros. 5. El ayuntamiento le asigna un presupuesto anual de 28.000 euros. 6. La sede de la agrupación está en el antiguo cuartel de la Guardia Civil. 7. Montejo tiene 7.900 habitantes.

Pregunta: ¿Qué actividad tuvo la agrupación de voluntarios de Montejo?

Respuesta de la IA: «La agrupación, con 46 voluntarios y 28.000 euros anuales, intervino en 2025 en tres incendios forestales gracias a su vehículo con depósito de 600 litros.»

> **Solución**
>
> **Alucina (tipo: no respaldado).** Todos los datos (46 voluntarios, 28.000 euros, tres incendios, el vehículo de 600 litros) están en las fuentes, pero **ninguna dice que interviniera «gracias a» ese vehículo**. La respuesta se inventa una relación de causa.
>
> Lo interesante: el juez de IA que os damos como señal dice que esta respuesta está **respaldada, con un 85 % de confianza**. Se equivoca. Vuestro bot tiene que ser más listo que él.

4. Qué recibís

4.1 El kit

Fichero	Para qué sirve
README.md	Las reglas, el formato de las señales, cómo entregar y cómo se puntúa.
EJEMPLOS.md	Casos comentados de cada tipo de alucinación y el criterio exacto de etiquetado.
ejemplo_bot.py	Un bot que ya funciona. Vuestro punto de partida.
evaluar_local.py	Mide vuestro bot sobre el entrenamiento con la misma métrica que el leaderboard, y os dice cuál sería vuestro mejor corte.
generar_predicciones.py	Genera el fichero que entregáis.
train.jsonl	1.707 casos de entrenamiento **con su etiqueta** (fiel o alucina, y el tipo).

4.2 Los datos

Conjunto	Casos	Cuándo	Etiquetas
Entrenamiento (train.jsonl)	1.707	Con el kit	Sí
Test (test_publico.jsonl)	440	Jueves, 10:00	No: es lo que tenéis que predecir
Giro (giro_publico.jsonl)	220	Jueves, 15:30	No

Todo está en español. Cada línea de los ficheros es un caso en formato JSON. **Ojo: el entrenamiento no es igual que el test.** Los 631 casos cuyo identificador empieza por `H` están escritos como el test: muchas fuentes, respuestas que combinan datos y alucinaciones sutiles. El resto es más fácil. Si medís y calibráis con todo el entrenamiento, os vais a confiar: medid sobre todo con los casos `H`.

4.3 Las señales: lo que la IA ya ha calculado por vosotros

Hemos pasado cada caso por tres modelos de IA y os damos sus resultados dentro de `caso["senales"]`. La respuesta viene troceada de dos formas: por **oraciones** y por **fragmentos** más pequeños (cortados por comas y conjunciones).

Señal	Qué es	De dónde sale
respaldo_por_fuente	Para cada fuente, de 0 a 1: cuánto respalda esa fuente, ella sola, a la frase.	Modelo NLI
contradiccion_por_fuente	Para cada fuente, de 0 a 1: cuánto la contradice.	Modelo NLI
respaldo_todas contradiccion_todas	Lo mismo, pero leyendo todas las fuentes juntas.	Modelo NLI
similitud_por_fuente	Para cada fuente, de 0 a 1: cuánto se parecen los textos (tema y palabras, no verdad).	Modelo de similitud
llm (por oración)	Veredicto de un LLM juez: «respaldada», «no_respaldada» o «contradicha», la fuente en la que se apoya y su confianza de 0 a 1.	LLM MiniMax

Un modelo **NLI** (*natural language inference*) es un modelo entrenado para una sola tarea: dado un texto A y una frase B, estimar si A respalda B, la contradice o ninguna de las dos.

> **Importante**
> Las señales **no son la respuesta**. Los modelos se equivocan, y no en los mismos sitios. El NLI da falsas alarmas cuando una respuesta correcta combina datos de varias fuentes, y se lía con los números. El LLM juez acierta mucho más, pero se fía demasiado: a veces da por buena una frase que añade algo que nadie ha dicho. **Vuestro trabajo es decidir cuándo creer a cada uno, cómo combinarlos y qué comprobaciones añadir vosotros.**

5. Qué cuenta como alucinación

5.1 Los cuatro tipos

Tipo	Qué es	Ejemplo (fuente → respuesta)
Distorsión	Dato equivocado: cifra, fecha, unidad, entidad, o un dato que sí está en las fuentes pero es de otro.	«A facturó 42 millones y B, 18» → «B facturó 42 millones»
Contradicción	Dice lo contrario que una fuente, muchas veces sin ningún «no».	«la demanda aumentó un 8 %» → «la demanda cayó un 8 %»
No respaldado	Añade algo plausible que ninguna fuente dice: una causa, un plazo, una atribución, un detalle.	«cerró en 2019» → «cerró en 2019 por la caída de ventas»
Sobregeneralización	Amplía el alcance de lo que dicen las fuentes.	«en tres hospitales» → «en los hospitales del país»

5.2 El criterio exacto

Una respuesta es **fiel** solo si **cada** cosa que afirma está respaldada por las fuentes, leídas en conjunto. Basta **una** afirmación sin respaldo para que sea alucinación. Lo que cuenta es lo que dicen las fuentes, no lo que es verdad en el mundo.

Una respuesta fiel SÍ puede…	Una respuesta fiel NO puede…
• Decir lo mismo con otras palabras. • Juntar datos de varias fuentes en una frase. • Dejarse información fuera. • Hacer cuentas exactas con las cifras de las fuentes (sumar, «el doble», 40 % = «cuatro de cada diez»). • Cambiar de unidad sin perder precisión (2.000 m = 2 km).	• Endurecer un verbo («vacunó» ≠ «inmunizó», «propuso» ≠ «aprobó»). • Quitar un matiz («más de 150.000» ≠ «150.000»). • Ampliar el alcance («algunos» ≠ «todos»). • Añadir causas, motivos o consecuencias. • Cambiar quién hizo qué.

6. Cómo se trabaja, paso a paso

1. **Probad el bot de ejemplo** sobre el entrenamiento:

```
python evaluar_local.py --bot ejemplo_bot.py --datos train.jsonl
```

   Os dirá su puntuación, cuántas falsas alarmas da, cuántas alucinaciones se le escapan y cuál sería su mejor corte.

2. **Copiad** ejemplo_bot.py a mi_bot.py y **mejoradlo**. Medid cada cambio con evaluar_local.py. Con --prefijo H medís solo sobre los casos que se parecen al test (y no entrenéis sobre los mismos casos con los que medís).
3. **Generad las predicciones** sobre el test cuando os lo entreguen:

```
python generar_predicciones.py --bot mi_bot.py --test test_publico.jsonl --salida predicciones.csv
```

4. **Entregad** predicciones.csv y mi_bot.py juntos. El organizador os devuelve vuestra puntuación y aparecéis en el leaderboard.
5. **Después del giro (15:30)**, generad un único fichero con los dos conjuntos:

```
python generar_predicciones.py --bot mi_bot.py --test test_publico.jsonl giro_publico.jsonl --salida predicciones.csv
```

Formato del fichero que entregáis: una línea `id,score` por caso, con coma como separador y **punto decimal** (`t0001,0.73`). No lo abráis y guardéis con Excel: lo convertiría a `;` y `0,73` y se rechazaría.

7. Cómo se puntúa

7.1 La métrica

El organizador convierte cada score en una decisión (**score ≥ 0,5 = alucina**) y calcula:

> **puntuación = 1 − (4 × alucinaciones que se escapan + falsas alarmas) / número de respuestas fieles**

- **1** significa no fallar ninguno.
- **0** es lo que saca un bot que marca todo como alucinación. No hace falta programar nada para eso, así que no tiene mérito.
- **Menos de 0** es peor que no hacer nada.

Dejar pasar una alucinación cuenta **4 veces** más que una falsa alarma, porque en un guardián real es mucho peor que una invención llegue al cliente que revisar una respuesta de más.

Ejemplo

El test tiene 265 respuestas fieles. Un bot que deja escapar 20 alucinaciones y da 40 falsas alarmas saca 1 − (4 × 20 + 40) / 265 = 1 − 120 / 265 = **0,55**.

Al lado de la puntuación se muestra el **AUROC**, que mide si vuestro bot ordena bien los casos (las alucinaciones por encima de las fieles) independientemente del corte. Sirve para desempatar.

> **Para que tengáis una referencia**
> El bot de ejemplo saca alrededor de −0,25 en el test, y fiarse a ciegas del juez LLM da −0,6: el juez da por buenas muchas alucinaciones sutiles. **Pasar de 0 ya tiene mérito, y por encima de 0,4 en la nota final es un resultado muy bueno.** No os desaniméis con números bajos al principio: es lo normal. El reto está pensado para que haya margen de mejora durante todo el día.

7.2 Parte pública y parte privada

El test y el giro están divididos en dos mitades que no conocéis. El leaderboard que veréis en la sala usa solo la mitad **pública**. La clasificación final se calcula con la mitad **privada**, a partir de vuestra entrega de después del giro con mejor nota pública, y se revela al cierre.

Por eso no sirve subir variantes y quedarse con la que más sube en el leaderboard: un bot que generaliza bien puntúa igual en las dos mitades.

7.3 Nota final

Parte	Peso	Cómo se calcula
Leaderboard	60 puntos	60 × vuestra puntuación final (parte privada de test + giro), con un mínimo de 0.
Jurado	40 puntos	Solo los 5 finalistas: pitch del viernes a las 9:30 (ver apartado 9).

8. Calendario

Cuándo	Qué pasa
Jueves 15 · 10:00	Presentación del reto. Recibís el kit y el test. Empezáis a entregar predicciones (**hasta 3 entregas** antes del giro).
Jueves · 14:15	Comida.
Jueves · 15:30	**El giro**: se publica una tanda nueva de casos con un formato distinto. A partir de aquí entregáis test + giro juntos (**3 entregas más**). Lo que solo funcionaba con el formato del entrenamiento se notará aquí.
Jueves · 19:00	Cierre de entregas. Se calcula la clasificación final con la parte privada.
Jueves · 19:00–20:15	Los 5 primeros ejecutan su bot sobre un lote nuevo delante del jurado (reproducción).
Jueves · 20:30	Anuncio de los 5 finalistas.
Viernes 16 · 9:30	Pitch de los finalistas (5 minutos) ante el jurado, en el Salón de Actos.
Viernes · 12:00	Entrega de premios y clausura.

9. Reglas y jurado

9.1 Reglas

- **Un bot por equipo**, en Python.
- **Podéis usar** las señales de cada caso, la librería estándar de Python, numpy y scikit-learn (por ejemplo, para entrenar un clasificador con el entrenamiento). Nada de eso es obligatorio: se puede ganar con Python puro.
- **No podéis usar** modelos de IA de ningún tipo: ni APIs externas (ChatGPT, Claude, Gemini…), ni modelos locales. Todos partís de las mismas señales, así que gana la estrategia, no el portátil ni el dinero.
- **Podéis leer** el test para entender el formato, pero **no podéis** etiquetarlo a mano ni meter en el código reglas pensadas para casos concretos del test.
- **Reproducción (jueves, 19:00–20:15):** los 5 primeros ejecutan su mi_bot.py delante del jurado sobre un lote nuevo que nadie ha visto. Un bot honesto saca una nota parecida. Si cae mucho, o el código no es el que se entregó, la entrega queda descalificada y entra el siguiente: quedaos hasta el anuncio de las 20:30.

9.2 Qué valora el jurado (40 puntos)

Criterio	Puntos	Qué se valora
Método	15	¿Cómo combináis las señales? ¿Sabéis cuándo desconfiar del juez LLM y del NLI? ¿Es un método sólido o un montón de reglas frágiles?
Calibración y giro	10	¿Elegisteis el corte sabiendo que un escape pesa 4 veces más? ¿Aguantó vuestro bot el cambio de las 15:30?
Originalidad	5	Ideas propias: cifras y fechas, a quién pertenece cada dato, alcance, combinación de señales.
Código y defensa	5	Código legible y una explicación clara de por qué funciona, o de por qué falla donde falla.
Propuesta de generalización	5	Cómo lo llevaríais a un guardián real: límites, siguiente paso, coste.

El jurado premia **entender** por encima de la posición: un equipo que explica con lucidez por qué su detector falla en ciertos casos demuestra más que uno que no sabe por qué acierta.

10. Pistas para empezar

- **Mirad el entrenamiento con calma.** Leed casos de cada tipo y comprobad qué dicen las señales en los que se equivocan.
- **El bot de ejemplo compara cada frase con cada fuente por separado.** Por eso falla con las respuestas correctas que combinan varias fuentes. Mirad también `respaldo_todas` y los fragmentos.
- **Una sola afirmación falsa basta.** Si promediáis las señales de todas las frases, la falsa se diluye. Buscad la peor.
- **El juez LLM no es infalible.** Pensad cuándo conviene desconfiar de un «respaldada»: poca confianza, el NLI no encuentra respaldo, la frase añade un «gracias a» o un «todos» que no está en la fuente que cita, la cifra no sale de las cuentas, el verbo es más fuerte que el de la fuente, desaparece un «podría» o un «solo»…
- **Cuidado con quién hizo qué.** Muchas fuentes hablan de dos entidades parecidas. Que un número aparezca en las fuentes no significa que esté bien atribuido.
- **No busquéis atajos.** Lo hemos comprobado: la longitud de la respuesta, los superlativos o las negaciones no predicen nada.
- **Calibrad el corte** con `evaluar_local.py`. Os dice cuál sería vuestro mejor corte; si no es 0,5, reescalad vuestro score para que lo sea.

11. Preguntas frecuentes

¿Necesito saber de inteligencia artificial?

No. Necesitáis manejaros con Python. Las señales ya traen el trabajo de los modelos; vosotros decidís qué hacer con ellas.

¿Necesito un ordenador potente o una tarjeta gráfica?

No. Basta cualquier portátil con Python 3.9 o superior. No se instala ni se ejecuta ningún modelo.

¿Puedo usar ChatGPT o Copilot para ayudarme a programar?

Sí, para escribir vuestro código. Lo que no puede hacer vuestro bot es llamar a ningún modelo de IA para decidir sobre los casos.

¿Cuántas personas por equipo?

Lo indicará la organización en la inscripción.

¿Qué pasa si mi fichero tiene un error de formato?

Se rechaza con un mensaje que explica el problema, y no gasta ninguna de vuestras entregas.

¿Puedo discutir una etiqueta?

Sí, ante el jurado. Se resuelve con el criterio escrito que tenéis en EJEMPLOS.md. Todas las etiquetas se han verificado dos veces por separado, y las dudosas se retiraron.